

World-Writable NLTK Cache Directory Enables Local Users to Tamper with or Delete NLP Data in run-llama/llama_index

✓ Valid

Reported on Jul 16th 2025

Description

The llama_index library sets the NLTK data directory to a subdirectory of the codebase by default (e.g., _static/nltk_cache inside the package directory). In multi-user environments or shared hosting, this directory is world-writable or accessible by multiple users. As a result, any user can overwrite, delete, or corrupt NLTK data files, leading to denial of service, data tampering, or potentially privilege escalation if a vulnerable data loader is present.

The issue is that this implementation does not use a user-specific cache directory, making it vulnerable to local data tampering and denial of service in shared environments.

issue present at - llama-index-core/llama_index/core/utils.py

vulnerable code block

```
def wait_for_nltk_check(self) -> None:
    """Initialize NLTK data download."""
    from nltk.data import path as nltk_path

    # Set up NLTK data directory
    self._nltk_data_dir = os.environ.get(
        "NLTK_DATA",
        os.path.join(
            os.path.dirname(os.path.abspath(__file__)),
            "_static/nltk_cache",
        ),
    )

    # Ensure the directory exists
    os.makedirs(self._nltk_data_dir, exist_ok=True)

    # Add to NLTK path if not already present
    if self._nltk_data_dir not in nltk_path:
        nltk_path.append(self._nltk_data_dir)

    # Start downloading NLTK data / confirming it's available
    self._download_nltk_data()
```

Example Patch

```
import platformdirs

def wait_for_nltk_check(self) -> None:
    """Initialize NLTK data download."""
    from nltk.data import path as nltk_path

    # Set up NLTK data directory (user-specific)
    self._nltk_data_dir = os.environ.get(
        "NLTK_DATA",
        os.path.join(
            platformdirs.user_cache_dir("llama_index"),
            "nltk_cache",
        ),
    )

    # Ensure the directory exists
    os.makedirs(self._nltk_data_dir, exist_ok=True)

    # Add to NLTK path if not already present
    if self._nltk_data_dir not in nltk_path:
        nltk_path.append(self._nltk_data_dir)

    # Start downloading NLTK data / confirming it's available
    self._download_nltk_data()
```

Similar patch

A similar patch was done to `get_cache_dir()` by the team, such that the `get_cache_dir()` function to always use a user-specific cache location via `platformdirs.user_cache_dir("llama_index")`. This change ensures that the main cache directory is no longer shared across users, reducing the risk of local data tampering or denial of service for general cache files.

```
import platformdirs

def get_cache_dir() -> str:
    # User override
    if "LLAMA_INDEX_CACHE_DIR" in os.environ:
        path = Path(os.environ["LLAMA_INDEX_CACHE_DIR"])
    else:
        path = Path(platformdirs.user_cache_dir("llama_index"))

    path.mkdir(parents=True, exist_ok=True)
    return str(path)
```

However: NLTK Data Directory Still Vulnerable

Despite this improvement, the NLTK data directory is still set to a subdirectory of the codebase by default (e.g., `_static/nltk_cache`), not the user-specific cache directory. This means the NLTK data files remain vulnerable to local tampering and denial of service in multi-user environments.

```
def wait_for_nltk_check(self) -> None:
    """Initialize NLTK data download."""
    from nltk.data import path as nltk_path

    # Set up NLTK data directory
    self._nltk_data_dir = os.environ.get(
        "NLTK_DATA",
        os.path.join(
            os.path.dirname(os.path.abspath(__file__)),
            "_static/nltk_cache",
        ),
    )

    # . . . . .
```

Impact

Data Tampering: Any user with filesystem access can overwrite or modify NLTK data files (e.g., stopwords, tokenizers), causing all other users of the library on the same system to load malicious or corrupted data. **Denial of Service:** Deleting or corrupting these files can cause all dependent code to fail or crash. **Potential Privilege Escalation:** If NLTK or related code is ever found to be vulnerable to code execution via malicious data files, this could be a vector for local privilege escalation. **Integrity and Availability Risks:** All users of the library on the same system are affected if the directory is world-writable or shared.

Occurrences

 `utils.py` L49-L70

⚙️ We are processing your report and will contact the [run-llama/llama_index](#) team within 24 hours. 6 months ago

🕒 A [GitHub Issue](#) asking the maintainers to create a `SECURITY.md` exists 6 months ago



huntr-helper commented 6 months ago

[Admin](#)

This report was determined to possibly be out of scope or have a high likelihood of being marked as informative.

Please review your report and the [Participation Guidelines](#).

These are the specific guidelines this report is in possible violation of:

- Issues related to outdated package dependencies or vulnerabilities within the package dependencies of the target



madan301 commented 6 months ago

Researcher

Hi team,

Thank you for reviewing my submission. I would like to clarify that this particular issue is not related to an outdated package or external dependency vulnerability.

The bug I reported is present in the project's own code, specifically in the `utils.py` file under `llama-index-core/llama_index/core/`. It relates to how the NLTK data directory is configured:

By default, the code sets the NLTK data directory to a shared location inside the package directory (e.g., `_static/nltk_cache`).

This location can become world-writable or accessible by multiple users in shared or multi-user environments (e.g., university servers, research clusters).

This makes it possible for any local user to delete, corrupt, or tamper with NLP resources used by others — leading to denial of service, data poisoning, or even local privilege escalation if a vulnerable data loader is involved.

This is a local misconfiguration vulnerability introduced by the package itself. It persists even after the team has patched similar issues in their `get_cache_dir()` function using `platformdirs.user_cache_dir()` — however, this fix was not extended to the NLTK data path.

To reiterate, this is not about an outdated version of NLTK, but rather about insecure default behavior in how `llama_index` integrates with it.

I'd appreciate it if you could kindly take another look at this issue, and I'm happy to provide additional clarification or a PoC if needed.

Thanks again!



Massimiliano Pippi validated this vulnerability 6 months ago

\$ **madan301** has been awarded the disclosure bounty ✓

\$ The fix bounty is now up for grabs

★ The researcher's credibility has increased: +7



CVE-2025-7707 assigned to this report. 6 months ago



madan301 commented 6 months ago

Researcher

Hi Team, I see the fix is done in the repo. Pls verify soon. Thank you.



Massimiliano Pippi marked this as fixed in **v0.13.0** with commit **988163** 6 months ago

\$ Massimiliano Pippi has been awarded the fix bounty ✓

\$ utils.py#L49-L70 has been validated ✓



madan301 commented 6 months ago

Researcher

Hi Team, thank you for the fix verification, appreciate it!, pls let me know about the CVE release. Thank you.



madan301 commented 6 months ago

Researcher

Hi team, awaiting for the CVE status, pls let me know. Thank you.



madan301 commented 5 months ago

Researcher

any update?

⚠ We have sent a warning to the **run-llama/llama_index** team to inform them that this report will be published in 48 hours 4 months ago

✉ We have notified the **run-llama/llama_index** maintainers about this report in their weekly follow-up 4 months ago



This vulnerability has now been published 4 months ago



CVE-2025-7707 has now been published 4 months ago

[Sign in](#) to join this conversation

CVE

CVE-2025-7707

Published

Vulnerability Type

CWE-377: Insecure Temporary File

Severity

High (7.1)

Attack vector	Local
Attack complexity	Low
Privileges required	Low
User interaction	None
Scope	Unchanged
Confidentiality	None
Integrity	High
Availability	High

Registry
Pypi

Affected Version
0.12.33

Visibility
Public

Status
Fixed

Disclosure Bounty
\$750

Fix Bounty
\$187.5

Found by



madan301
@madan301
NOISY


Fixed by



Massimiliano Pippi
@masci
UNPROVEN


Supported by [Palo Alto Networks](#) and Prisma AIRS, leading the way to greater AI security.

